

Bonus: HATEOAS and Marshmallow (CodeGrade)

 [_ \(https://github.com/learn-co-curriculum/python-p4-hateoas-and-marshmallow\)](https://github.com/learn-co-curriculum/python-p4-hateoas-and-marshmallow)  [_ \(https://github.com/learn-co-curriculum/python-p4-hateoas-and-marshmallow/issues/new\)](https://github.com/learn-co-curriculum/python-p4-hateoas-and-marshmallow/issues/new)

Learning Goals

- Build RESTful APIs that are easy to navigate and use in applications.
-

Key Vocab

- **Representational State Transfer (REST)**: a convention for developing applications that use HTTP in a consistent, human-readable, machine-readable way.
 - **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
 - **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
 - **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
 - **POST** : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
 - **PATCH** : an HTTP request method that signifies that the client is attempting to update a resource with new information.
 - **PUT** : an HTTP request method that signifies that the client is attempting to update a resource with new information contained in a complete record.
 - **DELETE** : an HTTP request method that signifies that the client is attempting to delete a resource.
-

Introduction

In our bonus lesson on REST philosophy, we briefly mentioned that a key element to creating a uniform and accessible interface is replacing indexed resources with their URLs. We call this concept Hypertext as the Engine of Application state, or **HATEOAS**. Because REST applications are stateless, with no knowledge of past requests to help guide users, URLs can be used to give those users the information that they need to find new resources from a starting resource.

This provides a bit of a problem on our end: how can we generate these URLs to implement HATEOAS in our application? There are several solutions available (including coding by hand!), but the best solution available to us in Flask is to switch to a serializer hyper-focused on HATEOAS: **Marshmallow**.

Marshmallow is not quite as hands-free as SQLAlchemy-Serializer, so it's important to figure out what functionality you need before you begin work on your application. A good rule of thumb when serializing in Flask: if you're using REST, use Marshmallow. If you want to define a unique format for your serialized data, use Marshmallow. If you don't need these extra features, SQLAlchemy-Serializer is simpler to implement and a good choice to get your app working quickly. Flask gives you a ton of options for extensions- remember to explore your options whenever you get started on a new app!

That being said, this is a lesson on HATEOAS- we're definitely using Marshmallow for that!

Setting up Flask-Marshmallow

Run `pipenv install; pipenv shell` to create and enter your virtual environment. In addition to the modules from earlier Phase 4 lessons, this will install two new libraries: Flask-Marshmallow and Marshmallow-SQLAlchemy.

Flask-Marshmallow will wrap our Flask application instance. Marshmallow-SQLAlchemy will actually be installed as an extension to Flask-Marshmallow and provide us with tools to automate some of the mappings between our models and serializer schema.

Enter the `server/` directory and enter the following commands to create and seed your database:

```
$ flask db upgrade
$ python seed.py
```

Now let's get to coding!

Setting up models

Using Marshmallow, our models are serialized *after* they've been generated. This means that we can remove all of the SQLAlchemy-Serializer code from `models.py` :

```
# server/models.py

from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class Newsletter(db.Model):
    __tablename__ = 'newsletters'

    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String)
    body = db.Column(db.String)
    published_at = db.Column(db.DateTime, server_default=db.func.now())
    edited_at = db.Column(db.DateTime, onupdate=db.func.now())

    def __repr__(self):
        return f'<Newsletter {self.title}, published at {self.published_at}>'
```

That's it! Marshmallow will require us to modify some code in our app in addition to writing a schema for serialization, but it stays out of our models entirely.

Configuring a Schema

Marshmallow decides how to present the data from your database according to a schema, or blueprint. This is a similar idea to a schema in SQL, but make sure you don't get them confused: a serializer's schema informs a server how to present data. A database schema informs a server how to store data.

Before defining our schema, we have to instantiate Marshmallow. This requires us to import Marshmallow and initialize with an instance of the Flask application.

IMPORTANT: A Marshmallow instance *must* be instantiated after our database. The interpreter will throw all sorts of errors if we do it before!

Enter the following in `server/app.py` , before the definitions of your routes:

```
# server/app.py

#!/usr/bin/env python3

from flask import Flask, request, make_response
from flask_marshmallow import Marshmallow
from flask_migrate import Migrate
from flask_restful import Api, Resource

from models import db, Newsletter

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///newsletters.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.json.compact = False

migrate = Migrate(app, db)
db.init_app(app)

ma = Marshmallow(app)
```

Not much new here- we're just importing some helpful libraries and instantiating Marshmallow with the Flask application instance we've created.

Next, we're going to configure our schema:

```
class NewsletterSchema(ma.SQLAlchemySchema):

    class Meta:
        model = Newsletter
        load_instance = True

    title = ma.auto_field()
    published_at = ma.auto_field()

    url = ma.Hyperlinks(
        {
            "self": ma.URLFor(
                "newsletterbyid",
                values=dict(id="<id>"),
            ),
            "collection": ma.URLFor("newsletters"),
        }
    )

newsletter_schema = NewsletterSchema()
newsletters_schema = NewsletterSchema(many=True)
```

Note that `NewsletterSchema` inherits from a `SQLAlchemySchema` parent class. This class is not necessary to use with Marshmallow, but it does allow us to autogenerate some attributes using SQLAlchemy's `Model` class. You can see these added below, with `title` and `published_at`. It is important to note here that this means that these are the only attributes that will appear when we look at newsletters- HATEOAS specifies that we should limit what is shown in collections- but that this can be overridden for single-record views with additional

schemas. For the time being, we're only going to write one. (It would be great practice to write the second on your own, though!) The statement `load_instance=True` is optional and indicates model instances should be deserialize.

Next, we set up URLs for single records and for the full collection on each record. This will help users- especially of the application variety- navigate our API. These are set for lowercase versions of our view class/function names.

Lastly, we instantiate the schema for single records and for multiple records. We will use these to serialize data in our views.

Displaying Serialized Data in Views

Now that we've made the switch from SQLAlchemy-Serializer to Marshmallow, we need to remove all of those `to_dict()` calls from our views. They will be replaced with the `schema.dump()` method, which will convert our records from SQL to JSON (no need for `jsonify`!). Let's look at one example below:

```
# server/app.py

class Newsletters(Resource):

    def get(self):

        newsletters = Newsletter.query.all()

        response = make_response(
            newsletters_schema.dump(newsletters),
            200,
        )

        return response
```

Here, we carry out most tasks as normal: create, retrieve, update, delete in the database with SQLAlchemy, then make a response object.

In making our response object, we use `newsletters_schema.dump()` to get the JSON for multiple newsletter records into the response object, then return it as normal. Running your server with `flask run`, you should see a list of newsletter titles and publication times with URLs for their single records and the full list of newsletter records.

Take some time to move all of your views to this format- remove every instance of `to_dict()` and replace with `schema.dump()` to show our new, HATEOAS-compliant views to our user base. (Full solution code is available below.)

Conclusion

HATEOAS is an important component of a uniform interface in RESTful APIs. It requires a bit of extra work to configure our views to show hyperlinks, but they are crucial in helping users navigate the API. Marshmallow is a serialization tool designed to help developers implement HATEOAS into their applications, and will be an important tool in streamlining the creation of RESTful APIs in your career.

Solution Code

```
#!/usr/bin/env python3

from flask import Flask, request, make_response
from flask_marshmallow import Marshmallow
from flask_migrate import Migrate
from flask_restful import Api, Resource

from models import db, Newsletter

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///newsletters.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.json.compact = False
```

```
migrate = Migrate(app, db)
db.init_app(app)
```

```
ma = Marshmallow(app)
```

```
class NewsletterSchema(ma.SQLAlchemySchema):
```

```
    class Meta:
```

```
        model = Newsletter
        load_instance = True
```

```
    title = ma.auto_field()
```

```
    published_at = ma.auto_field()
```

```
    url = ma.Hyperlinks(
```

```
        {
```

```
            "self": ma.URLFor(
                "newsletterbyid",
                values=dict(id="<id>")),
```

```
            "collection": ma.URLFor("newsletters"),
```

```
        }
```

```
    )
```

```
newsletter_schema = NewsletterSchema()
```

```
newsletters_schema = NewsletterSchema(many=True)
```

```
api = Api(app)
```

```
class Index(Resource):
```

```
    def get(self):
```



```
    response_dict = {
        "index": "Welcome to the Newsletter RESTful API",
    }

    response = make_response(
        response_dict,
        200,
    )

    return response

api.add_resource(Index, '/')

class Newsletters(Resource):

    def get(self):

        newsletters = Newsletter.query.all()

        response = make_response(
            newsletters_schema.dump(newsletters),
            200,
        )

        return response

    def post(self):

        new_newsletter = Newsletter(
            title=request.form['title'],
            body=request.form['body'],
        )
```

```
db.session.add(new_newsletter)
db.session.commit()

response = make_response(
    newsletter_schema.dump(new_newsletter),
    201,
)

return response
```

```
api.add_resource(Newsletters, '/newsletters')
```

```
class NewsletterByID(Resource):
```

```
    def get(self, id):
```

```
        newsletter = Newsletter.query.filter_by(id=id).first()
```

```
        response = make_response(
            newsletter_schema.dump(newsletter),
            200,
        )
```

```
        return response
```

```
    def patch(self, id):
```

```
        newsletter = Newsletter.query.filter_by(id=id).first()
        for attr in request.form:
            setattr(newsletter, attr, request.form[attr])
```

```
        db.session.add(newsletter)
        db.session.commit()
```

```
response = make_response(
    newsletter_schema.dump(newsletter),
    200
)

return response

def delete(self, id):

    record = Newsletter.query.filter_by(id=id).first()

    db.session.delete(record)
    db.session.commit()

    response_dict = {"message": "record successfully deleted"}

    response = make_response(
        response_dict,
        200
    )

    return response

api.add_resource(NewsletterByID, '/newsletters/<int:id>')

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Resources

- **Flask-Marshmallow**  (<https://flask-marshmallow.readthedocs.io/en/latest/>)

This tool needs to be loaded in a new browser window

Load Bonus: HATEOAS and Marshmallow (CodeGrade) in a new window